Seminar

Engineering Scalable Systems: The Core Principles of LLD and SOLID

Presented by:

Prabhu Teja Pamula

24EMCA1237

Seminar Guide:

Dr. Brijendra Singh

Associate Professor

Final Viva
10 July 2026

Outline

1. Architectural Assessment & Problem Domain
2. The OOP to SOLID Transition
3. The SOLID Engineering Framework
4. Design Patterns & Real World Case Studies
5. Conclusion & Takeaways

Phase 1

Architectural Assessment & Problem Domain

Problem Domain (Why Code Breaks)

The Big Mistake:

Many developers think that if their code runs without errors today, it is perfect.

Rushed Work:

When deadlines are tight, developers write messy code just to get the job done quickly.

Technical Debt:

Taking shortcuts is like borrowing money. It helps you finish fast today, but you spend all your time fixing bugs later.

The Result:

A simple change that should take one hour ends up taking days because the code is a mess.

Rigid vs. Fragile Code

Rigid Code (Too Stiff):

The code is so stiff that making a small change in one file forces you to change ten other files.

Fragile Code (Too Unstable):

The system breaks easily. You fix a bug in the Login screen, and the Payment screen randomly crashes.

High Coupling (Glued Together):

This happens because different parts of the code are tightly glued together instead of being independent pieces.

High Level vs. Low Level Design

High Level Design (HLD):

This is the big map of the project. It deals with server hosting, networks, and choosing our database.

Low Level Design (LLD):

This is the internal code level. It defines exactly how our coding classes and files talk to each other.

The Main Boundary:

Good server infrastructure cannot save a project if the underlying code is a messy, unmanageable mix.

Phase 2

The OOP to SOLID Transition

The Paradox of OOP

The Inheritance Trap:

Many developers use inheritance as a lazy shortcut just to reuse code, which is a mistake.

The Parent-Child Issue:

A tiny change in the top parent class can easily break dozens of child classes down the line.

Bad Encapsulation:

Creating public Getters and Setters for every single variable leaks data and ruins code security.

Code Design Smells

What is a Smell?:

A design smell is not a compiler error or a crash. It is a warning sign that your code structure is getting messy.

Stiff Code (Inelasticity):

The code is hard and stubborn. It refuses to stretch or change when you try to add a new feature.

Trapped Code (Immobility):

A good feature, like a login screen, is so glued to local files that you cannot copy and reuse it in a new project.

The Lazy Way (Viscosity):

The coding environment makes it faster and easier for a developer to write a messy hack than to write clean code.

Phase 3

The SOLID Engineering Framework

Introduction to SOLID Principles

The Missing Rules:

Basic OOP gives us coding blocks, but it does not give us standard rules on how to safely connect them.

The Main Goal:

The entire point of the SOLID framework is to limit the blast radius of a code change.

Long Term Speed:

It ensures that adding a new feature or fixing a bug in one place never ripples outward to break unrelated modules.

Single Responsibility Principle (SRP)

The Basic Rule:

A single coding class should be responsible to one, and only one, business user or actor.

The Mistake:

Creating a giant "God Class" that handles payroll for Finance, saves data for DBAs, and makes reports for HR.

The Fix:

We break it into three separate files so a change requested by Finance never accidentally breaks HR's reporting code.

Open/Closed Principle (OCP)

The Main Idea:

Your software files should be open for extension, but completely closed for modification.

Closed for Modification:

Once a core code file is written, tested, and running in production, you should never touch or alter its existing code to add a new feature.

Open for Extension:

To add a brand-new feature, you should write an entirely new, separate file or class that plugs into the old code safely.

Liskov Substitution Principle (LSP)

The Main Idea:

Any replacement part you plug into a system must work exactly like the original part without causing a breakdown.

The Rule:

A child piece of code must honor all the rules and expectations set by its parent piece of code.

The Mistake:

You have a parent class called PaymentProcessor with a process() function. If you add a new child class called CashPayment that throws a NotSupportedException because cash cannot be processed online, you break LSP and crash the app.

Interface Segregation Principle (ISP)

The Main Idea:

A class should never be forced to depend on or implement coding methods that it does not use.

The Mistake (Interface Pollution):

You create a giant interface called Worker with two functions: work() and eat(). When you build a Robot class that uses this interface, you are forced to write an empty eat() function that the robot doesn't need.

The Clean Fix:

We slice the giant interface into two smaller, specific interfaces: Workable and Feedable. The Robot class now only implements Workable, keeping it lightweight and free of junk code.

Dependency Inversion Principle (DIP)

The Main Idea:

High level code logic should not depend directly on low-level code details. Both should depend on abstract interfaces.

The Mistake (Tight Coupling):

A high level PasswordReset class directly depends on a low level GmailService class to send emails. If you decide to switch to OutlookService next month, you have to rewrite your core password reset logic, which risks introducing bugs.

The Clean Fix:

We introduce an EmailSender interface in the middle. Now, PasswordReset only talks to the interface, and both GmailService and OutlookService implement that interface. This decouples the classes entirely.

Phase 4

Design Patterns & Real World Case Studies

Strategy Design Pattern (Behavioral)

The Category:

Behavioral Pattern, focuses on how objects communicate and swap responsibilities at runtime.

The Main Idea:

Defines a family of interchangeable algorithms and encapsulates each one inside a separate class so they can be swapped dynamically.

The Mistake:

Using giant, nested if-else or switch statements inside a core class to handle different behaviors, like calculating shipping costs for different regions. Adding a new region forces you to rewrite and re-test the entire class.

The Clean Fix:

Create a common ShippingStrategy interface and build separate classes like USShipping and EUShipping that implement it. The core system just calls the interface, letting you add new regions without touching old code.

Factory Method Pattern (Creational)

The Category:

Creational Pattern, focuses on how objects are created safely behind the scenes.

The Main Idea:

Provides a centralized way to create objects without exposing the exact instantiation logic to the main application.

The Mistake:

Hardcoding the new keyword inside your business logic to create database connections (like new MySQLDatabase()). If the company switches to MongoDB, you have to hunt down and manually change every single file that hardcoded that database creation.

The Clean Fix:

Create a central DatabaseFactory class. The main system simply asks this factory to give it a database connection. Changing the database type now only requires updating one line of code inside the factory.

Observer Design Pattern (Behavioral)

The Category:

Behavioral Pattern, focuses on real time decoupling and how objects notify each other about state changes.

The Main Idea:

Defines a one-to-many dependency where one central object (the Subject) automatically notifies all of its registered dependents (the Observers) whenever its state changes.

The Mistake:

Tightly coupling notification tasks inside your core logic. For example, when a user registers, the UserAccount class directly calls EmailService.send(), SmsService.send(), and DatabaseLogger.log(). If you want to add mobile Push Notifications later, you must rewrite and re test the core registration class.

The Clean Fix:

Create an Observer interface that all notification services implement. The UserAccount class simply keeps a list of these observers and loops through them to notify them. You can plug in new notification channels (like Push Notifications) dynamically without changing the core registration file.

Conclusion & Takeaways

The Big Picture:

Implementing SOLID principles and Design Patterns isn't about writing more code, it is about writing code that is cheap to change and easy to maintain.

Key Project Result:

By decoupling our components (like using Factories and Strategies), we transformed a rigid codebase into an extensible system where new features can be added without breaking old ones.

The Golden Rule:

Never over engineer on day one. Use these principles when your code starts growing, when a file gets too large, or when you notice tight coupling between classes.

Thank You